CS520: Analysis of algorithms

Ewan Davies

Colorado State University

Fall 2026

Algorithms via local occupancy

Where we left off

- $G = (V, E)$ is a finite simple graph, $n = |V|$, and $d(u) \leq \Delta$
- $\mathcal{I}(G)$ is the set of independent sets, including $\emptyset$
- A hard-core random independent set $X \sim \pi$ at fugacity $\lambda > 0$ has law

$$\pi(I) = \frac{\lambda^{|I|}}{Z_G(\lambda)}, \qquad Z_G(\lambda) = \sum_{I \in \mathcal{I}(G)} \lambda^{|I|}$$

- Write $p = \lambda/(1+\lambda)$

The local occupancy hypothesis

Recall that local (β, γ)-occupancy, with $\beta, \gamma > 0$, means that for every u and every induced subgraph F of $G[N(u)]$,

$$\beta p \frac{1}{Z_F(\lambda)} + \gamma \frac{\lambda Z_F'(\lambda)}{Z_F(\lambda)} \geq 1.$$

- Abbreviate $E_F(\lambda) = \lambda Z_F'(\lambda)/Z_F(\lambda)$ and $\Gamma = \beta + \gamma\Delta$
- This includes $F = \emptyset$, for which $Z_F = 1$ and $E_F = 0$
- Then we must always have $\beta p \geq 1$
- Conditioning on the exterior and summing over u give $\alpha(G) \geq \mathbb{E}_\pi |X| \geq \frac{n}{\Gamma}$

Algorithms

- Existence of a large independent set does not tell us how to *find one efficiently*
- Listing all independent sets may take exponential time
- We will use simple random updates instead, inspired by physics
- For any $0 < \epsilon < 1$, our goal is to give an algorithm that, in time polynomial in n outputs a random independent set X^{alg} satisfying

$$\mathbb{E}|X^{\text{alg}}| \geq (1 - \epsilon)\frac{n}{\Gamma}$$

- The expectation is over the algorithm's randomness

Intuitive dynamics

Glauber dynamics: resample a random vertex

Start with any independent set, for example $X = \emptyset$. Repeat:

1. Choose a vertex u uniformly at random from V
2. If $X \cap N(u) \neq \emptyset$, leave u unoccupied
3. Otherwise: put u in X with probability p, and remove it with probability $1 - p$

- Everything outside u stays fixed
- An occupied vertex can be removed

Why this algorithm?



- It's a beautifully simple algorithm, you could probably code it in 5 lines of Python (maybe 15 of Lean…)
- It's essentially what happens in Physics
- If the vertices represent gas particles in some medium then by observing the medium at equilibrium as gas permeates in, out, and through it, we will essentially see this algorithm take place in continuous time
- Discrete time is a very nice simplification though!

Why this particular probability?

- Fix a current state I and its exterior $S = I \setminus \{u\}$
- If a neighbor is occupied, independence forces u to be absent
- Otherwise the two choices have hard-core weights in the ratio

$$\frac{\lambda^{|S|+1}}{\lambda^{|S|}} = \lambda$$

- Normalizing the two weights $1, \lambda$ gives probabilities $1 - p, p$
- Thus we resample u from its **conditional hard-core distribution**. This is called a *heat-bath update*

Example: the path $a - u - b$

At $\lambda = 1$, the coin is fair.

- From $I = \{u\}$, an update at a or b cannot add anything
- An update at u removes it with probability $1/2$
- Once u is absent, updates can add a and b, reaching $\{a, b\}$
- Escaping a small set can require first making it smaller
- This explains why we allow removals

Neighborhood dynamics: reconsider a whole region

Choose a uniform random vertex $u \in V$. Perform two updates in this order:

1. Keep $V \setminus N(u)$ fixed, and resample the **entire neighborhood** $N(u)$ from its conditional hard-core distribution
2. Then perform the single-vertex heat-bath update at u

- The center u is part of the fixed exterior during step 1
- In particular, if u is occupied, all its neighbors are forced to stay absent during that step
- This ordered pair of updates counts as **one block step**

How to resample a neighborhood

- For a fixed current state I, let $S = I \setminus N(u)$ be the exterior
- Delete from $N(u)$ every vertex having a neighbor in S; let F be the graph induced by the remaining available vertices
- Sample X_F with $\mathbb{P}(X_F = J) = \lambda^{|J|} / Z_F(\lambda)$ for $J \in \mathcal{I}(F)$, and replace the old neighborhood by X_F
- On a general graph, enumerate the at most $2^{d(u)}$ subsets to implement this in $2^{O(\Delta)}$ time

An algorithm that is easy to run

For a prescribed number T of steps:

1. Set $X_0 = \emptyset$ and remember it as the best set
2. Run either dynamics, using fresh randomness at each step
3. Whenever a larger set appears, remember it
4. Return the largest set visited

How large T should be so that we're likely to see a large independent set?

Markov chains

The probability space

- Our state space is $\Omega = \mathcal{I}(G)$
- A state $I \in \Omega$ is a particular independent set
- A probability distribution μ is a list of numbers $\mu(I) \geq 0$ summing to 1
- The point mass δ_I assigns probability 1 to the state I
- For a random set $X \sim \mu$, we have $\mathbb{P}(X = I) = \mu(I)$ and

$$\mathbb{E}_\mu f = \mathbb{E}[f(X)] = \sum_{I \in \Omega} \mu(I) f(I)$$

for any function $f : \Omega \to \mathbb{R}$, such as $f(I) = |I|$

A Markov chain remembers its current state

- A **Markov chain** is a sequence $X_0, X_1, \ldots$ whose next-state distribution depends only on the current state:

$$\mathbb{P}(X_{t+1} = J \mid X_0, \ldots, X_t) = \mathbb{P}(X_{t+1} = J \mid X_t)$$

- We study **time-homogeneous Markov chains** which use the same update rule at every time
- That is, we define

$$K(I, J) = \mathbb{P}(X_{t+1} = J \mid X_t = I)$$

- The **transition kernel** K is a matrix indexed by states, with nonnegative entries and each row summing to 1

Kernels multiply distributions from the right



- We regard a distribution μ over Ω as a **row vector**
- After an update with kernel K, μ becomes μK where

$$(\mu K)(J) = \sum_{I \in \Omega} \mu(I) K(I, J).$$

- Write μ_t for the distribution of X_t. Then

$$\mu_{t+1} = \mu_t K, \qquad \mu_t = \mu_0 K^t$$

- That means taking steps of the Markov chain corresponds to **matrix multiplication**

But still no efficient algorithms



- This does not, however, yield an efficient algorithm because the matrix can be exponential in size
- K has one row and column per independent set
- But we can perform certain transition kernels, like Glauber dynamics, without generating the full matrix ever!

A complete small transition matrix

For a single edge ab, order the states as $\emptyset, \{a\}, \{b\}$. Ordinary Glauber dynamics has matrix

$$K = \begin{pmatrix} 1-p & p/2 & p/2 \\ (1-p)/2 & (1+p)/2 & 0 \\ (1-p)/2 & 0 & (1+p)/2 \end{pmatrix}.$$

- For example, from $\{a\}$ we reach $\emptyset$ only by choosing a and then proposing absence
- Choosing the blocked vertex b leaves $\{a\}$ unchanged
- Check that each row sums to 1

Composing updates

- If we first apply A, then B, their kernel is the product AB:

$$(AB)(I,J) = \sum_{L \in \Omega} A(I,L)B(L,J)$$

- Thus μAB means **first A, then B**
- Kernels need not commute: AB and BA can describe different experiments
- If we choose one of $K_1, \ldots, K_n$ uniformly, the kernel is

$$K = \frac{1}{n} \sum_{u=1}^{n} K_u$$

Names for our update kernels

- P_u: resample the single vertex u
- Q_u: resample $N(u)$, keeping its exterior, including u, fixed
- Ordinary random-scan Glauber dynamics has kernel

$$K^G = \frac{1}{n} \sum_{u \in V} P_u$$

- Our neighborhood block dynamics has kernel

$$K^N = \frac{1}{n} \sum_{u \in V} Q_u P_u$$

- In $Q_u P_u$, the neighborhood is updated **before** the center

Kernels also act on functions

View a function $f : \Omega \to \mathbb{R}$ as a column vector. Then standard matrix multiplication gives

$$(Kf)(I) = \sum_J K(I, J) f(J).$$

- This is the expected value of f after applying K to state I
- Matrix multiplication gives

$$\mathbb{E}_{\mu K} f = \mu K f = \mathbb{E}_{\mu}(Kf)$$

Two observables we will track

- Define $x_u(I) = \mathbf{1}_{\{u \in I\}}$ and $y_u(I) = |I \cap N(u)|$
- Thus $\mathbb{E}_\mu x_u = \mathbb{P}_\mu(u \in X)$ and $\mathbb{E}_\mu y_u$ is the expected number of occupied neighbors of u
- The update rule at u gives

$$(P_u x_u)(I) = p\mathbf{1}_{\{y_u(I)=0\}}$$

- This is p when the neighborhood is empty, and 0 otherwise

Stationarity: a distribution preserved by an update

A distribution μ is **stationary** for K if $\mu K = \mu$.

- A heat-bath update preserves the hard-core model π: it keeps the exterior independent set fixed and replaces the interior by its conditional law under π
- In fact, we can verify that π is stationary for Q_u too
- Hence $\pi P_u = \pi Q_u = \pi$, so also $\pi K^G = \pi K^N = \pi$

Aside: stationarity

- Traditionally, Markov chain algorithms are designed to **approximate** a given distribution
- Given π, we design a kernel K such that $\pi K = \pi$
- We want this kernel to admit an efficient implementation
- The fundamental theorem of Markov chains gives us conditions under which, for any initial distribution μ we have $\lim_{t \to \infty} \mu K^t = \pi$
- Sometimes we can prove that after **polynomially many** steps $T = n^c$ we have that μK^T and π are **close**, this is called **rapid mixing**
- Rapid mixing for Glauber dynamics of the hard-core model is very well-understood [GaŠtVi16] [SlSu14] [AnLiOv20] [CFYZ22] [ChEl25]

Aside: optimization

- A more recent development is a framework for analyzing when slow-mixing Markov chains are still useful [LMRR24] [LMRR25]
- We are focusing on this idea here, ignoring any question of rapid mixing
- That is, we are answering the question of when very simple Markov chain algorithms like Glauber dynamics are good *optimization* algorithms
- Local occupancy already existed, and it's elegant to build a very simple algorithmic framework around it
- The framework goes way beyond independent sets…

References

[AnLiOv20] Anari, N., Liu, K., Oveis Gharan, S. (2020). *Spectral independence in high-dimensional expanders and applications to the hardcore model*. 2020 IEEE 61st annual symposium on foundations of computer science (FOCS), 1319–1330. DOI: https://doi.org/10.1109/FOCS46700.2020.00125

[CFYZ22] Chen, X., Feng, W., Yin, Y., Zhang, X. (2022). *Rapid mixing of glauber dynamics via spectral independence for all degrees*. 2021 IEEE 62nd annual symposium on foundations of computer science (FOCS), IEEE, 137–148. DOI: https://doi.org/10.1109/FOCS52979.2021.00022

[ChEl25] Chen, Y., Eldan, R. (2025). *Localization schemes: A framework for proving mixing bounds for Markov chains*. Duke Mathematical Journal, Duke

References

University Press, 174 (8), 1431–1510. DOI: https://doi.org/10.1215/00127094-2024-0063

[GaŠtVi16] Galanis, A., Štefankovič, D., Vigoda, E. (2016). *Inapproximability of the partition function for the antiferromagnetic ising and hard-core models*. Combinatorics, Probability and Computing, Cambridge University Press, 25 (4), 500–559. DOI: https://doi.org/10.1017/S0963548315000401

[LMRR24] Liu, K., Mohanty, S., Raghavendra, P., Rajaraman, A., Wu, D. X. (2024). *Locally stationary distributions: A framework for analyzing slow-mixing markov chains**. 2024 IEEE 65th annual symposium on foundations of computer science (FOCS), 203–215. DOI: https://doi.org/10.1109/FOCS61266.2024.00022

References

[LMRR25] Liu, K., Mohanty, S., Raghavendra, P., Rajaraman, A., Wu, D. X. (2025). *Locally Stationary Distributions: A Framework for Analyzing Slow-Mixing Markov Chains*. arXiv. DOI: https://doi.org/10.48550/arXiv.2405.20849

[SlSu14] Sly, A., Sun, N. (2014). *Counting in two-spin models on d-regular graphs*. The Annals of Probability, 42 (6), 2383–2416. DOI: https://doi.org/10.1214/13-AOP888